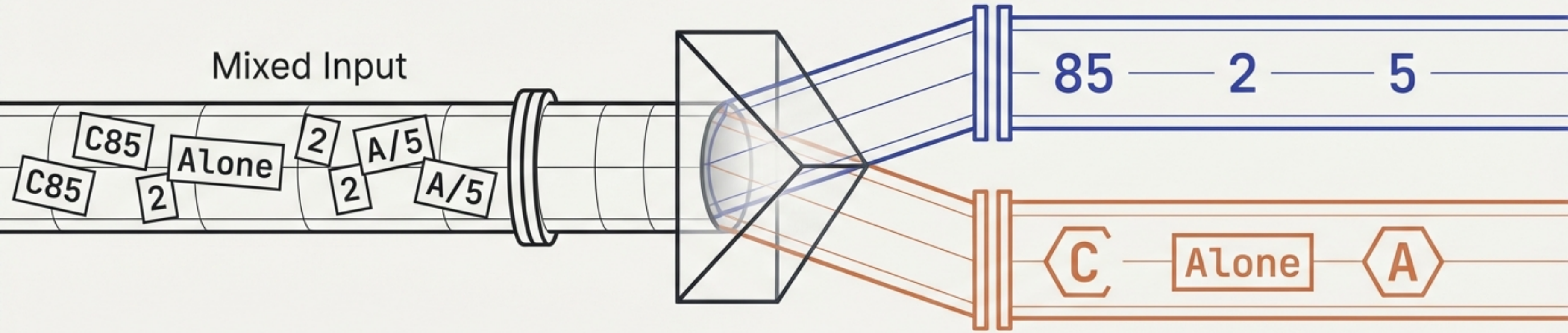


Mastering Mixed Variables

Advanced Feature Engineering Techniques

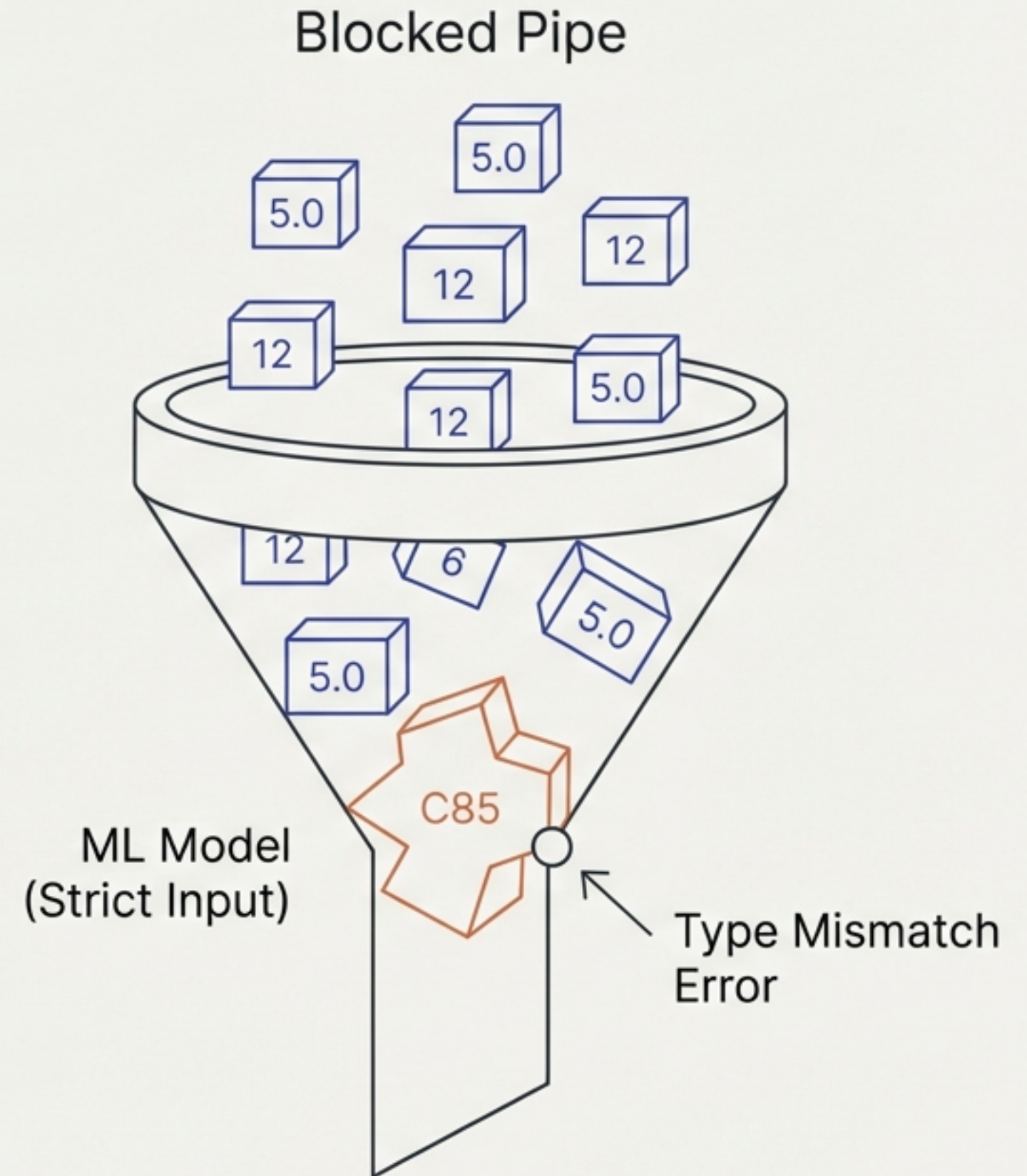


A field guide to untangling columns containing both numerical and categorical data.

Based on techniques for handling mixed types in Pandas.

The Pain Point: Mixed variables break strict models.

- **Definition:** Mixed Data occurs when a single column contains both numerical and categorical information.
- **The Conflict:** Machine Learning algorithms (like Scikit-Learn estimators) demand strict types. They cannot process ambiguity.
- **The Risk:** Ignoring this leads to `ValueError` crashes or significant data loss if rows are simply dropped.



Taxonomy: The two faces of mixed data.

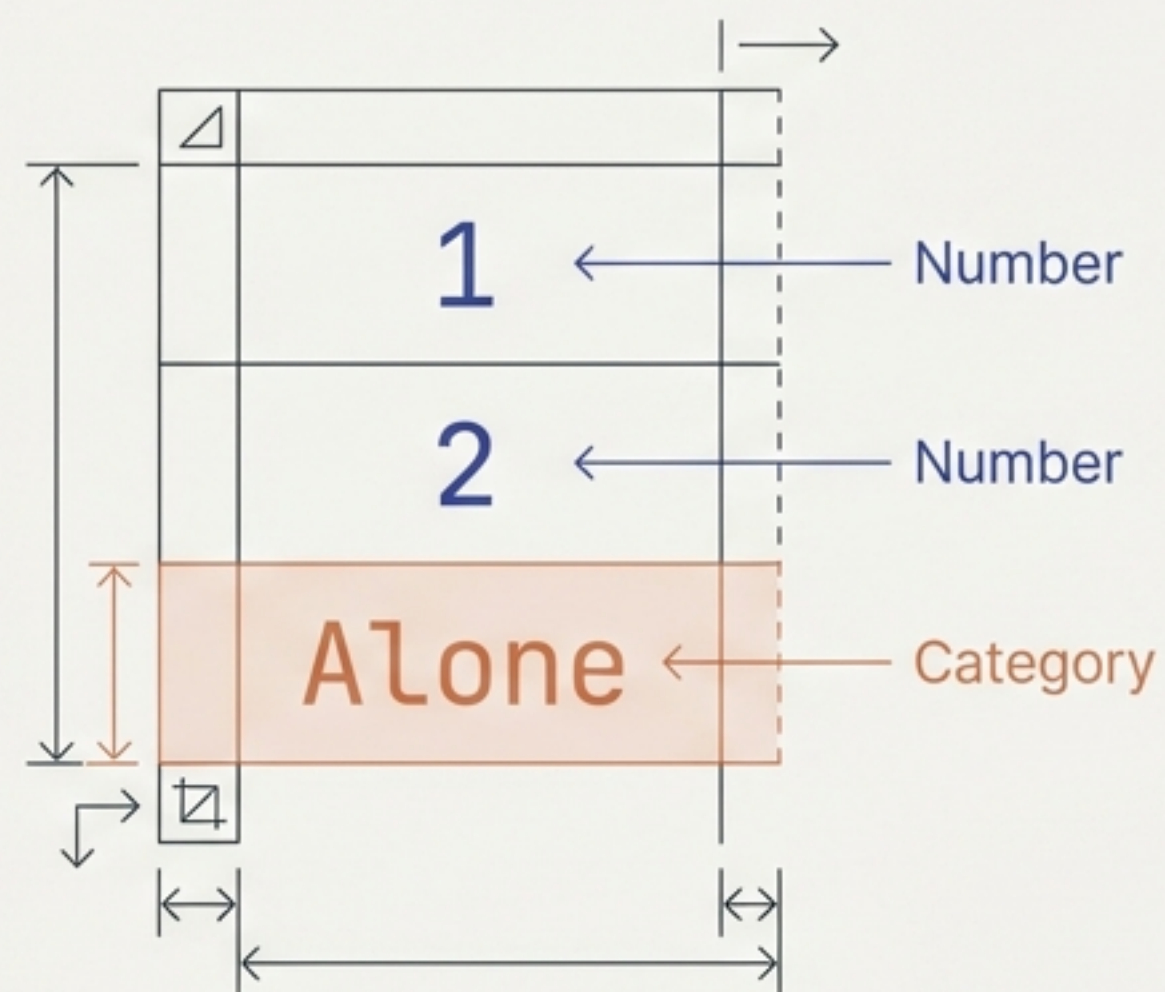
Scenario A: The Fused Mix

Information fused in a single cell.



Scenario B: The Interspersed Mix

Information varies by row.



Deep Dive: Scenario A (The Fused Mix)

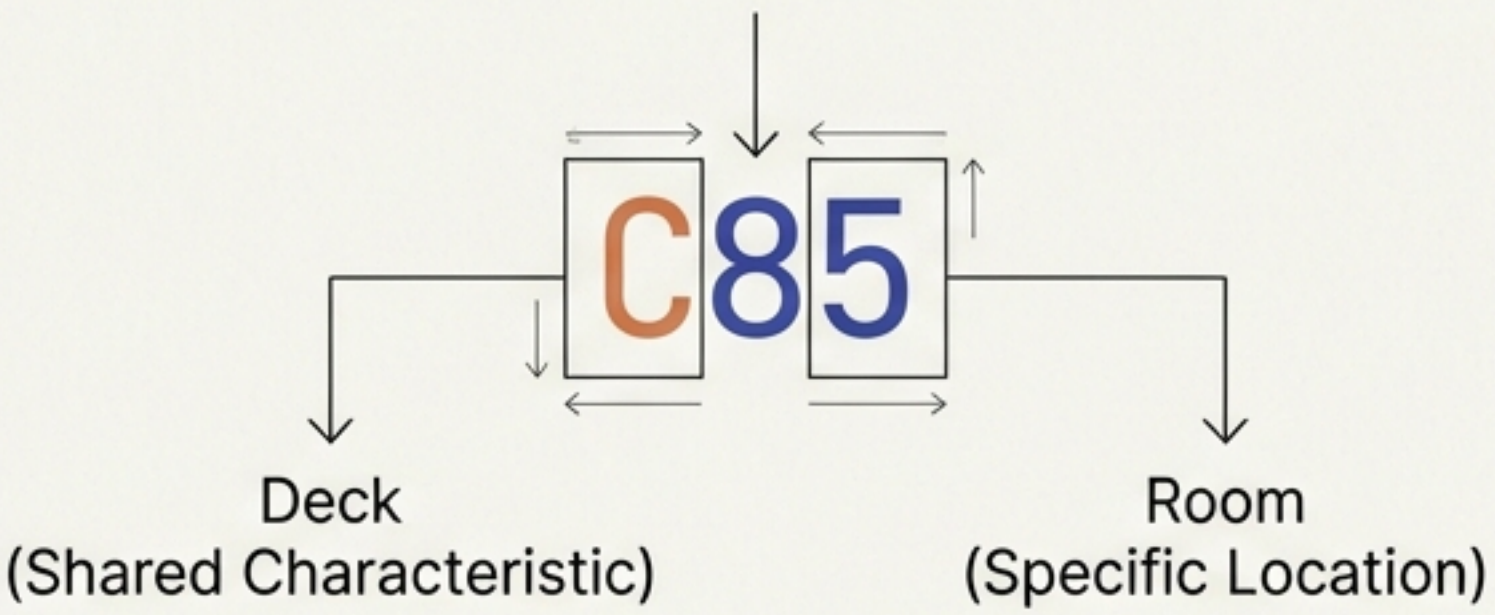
Case Study: The Titanic Dataset

Raw Data

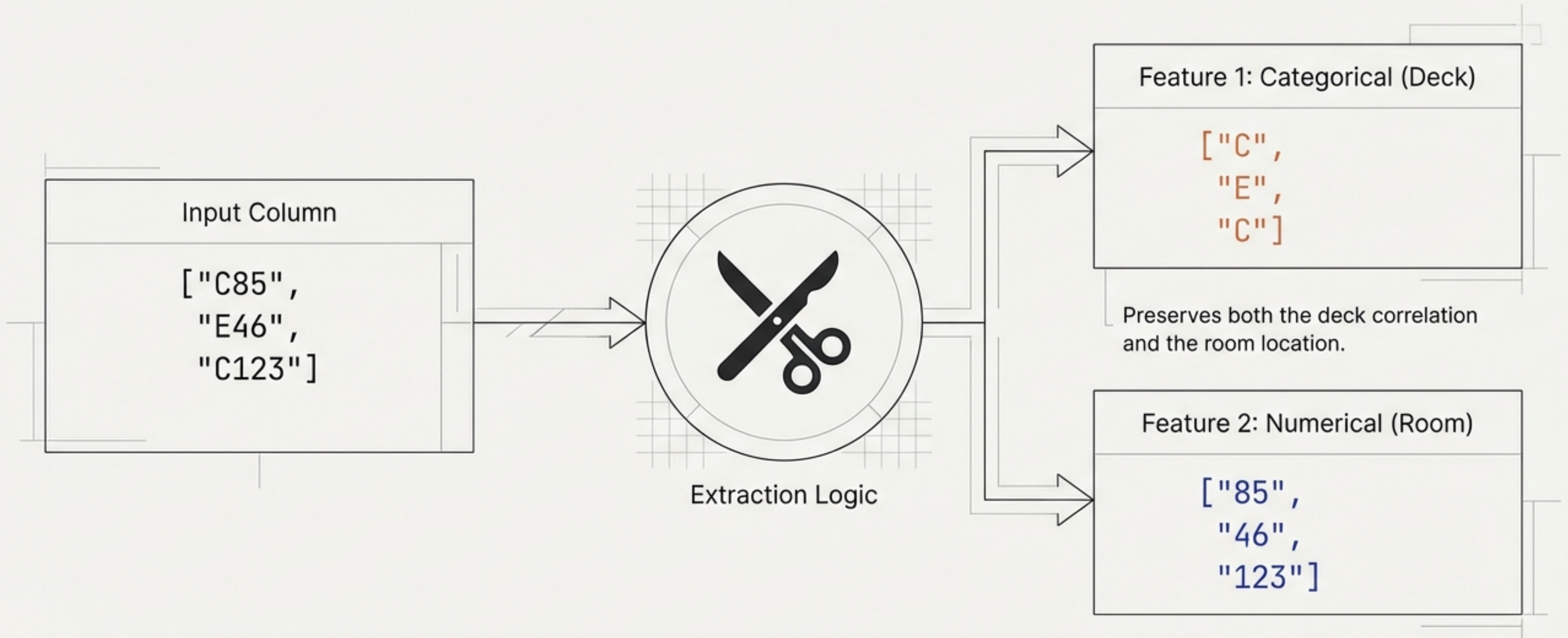
	PassengerId	Cabin	Ticket
1	1	C85	A/5 21171
2	2	C123	PC 17599
3	3	E46	STON/02. 3101282
4	4	G6	113803

Deconstruction

Problem: High Cardinality. If treated as a category, "C85" and "C86" are totally different labels. The model misses the pattern that they are both on Deck C.



The Strategy: Split and Extract



Execution: Regex Extraction

```
1 # 1. Extract the Category (First letter)
2 df['cabin_cat'] = df['Cabin'].str[0]
3
4 # 2. Extract the Number (Regular Expression)
5 # looks for digits (\d+) and captures them
6 df['cabin_num'] = df['Cabin'].str.extract('(\d+)')
```

Simple string indexing

Captures continuous digits

Transformed DataFrame

Cabin (Original)	cabin_cat	cabin_num
C85	C	85
E46	E	46
G6	G	6

Deep Dive: Scenario B (The Interspersed Mix)

Case Study: The 'Alone' Column.

Raw Column: Companions
1
2
Alone ⚠
5
Alone ⚠
3

The Pollution Problem:

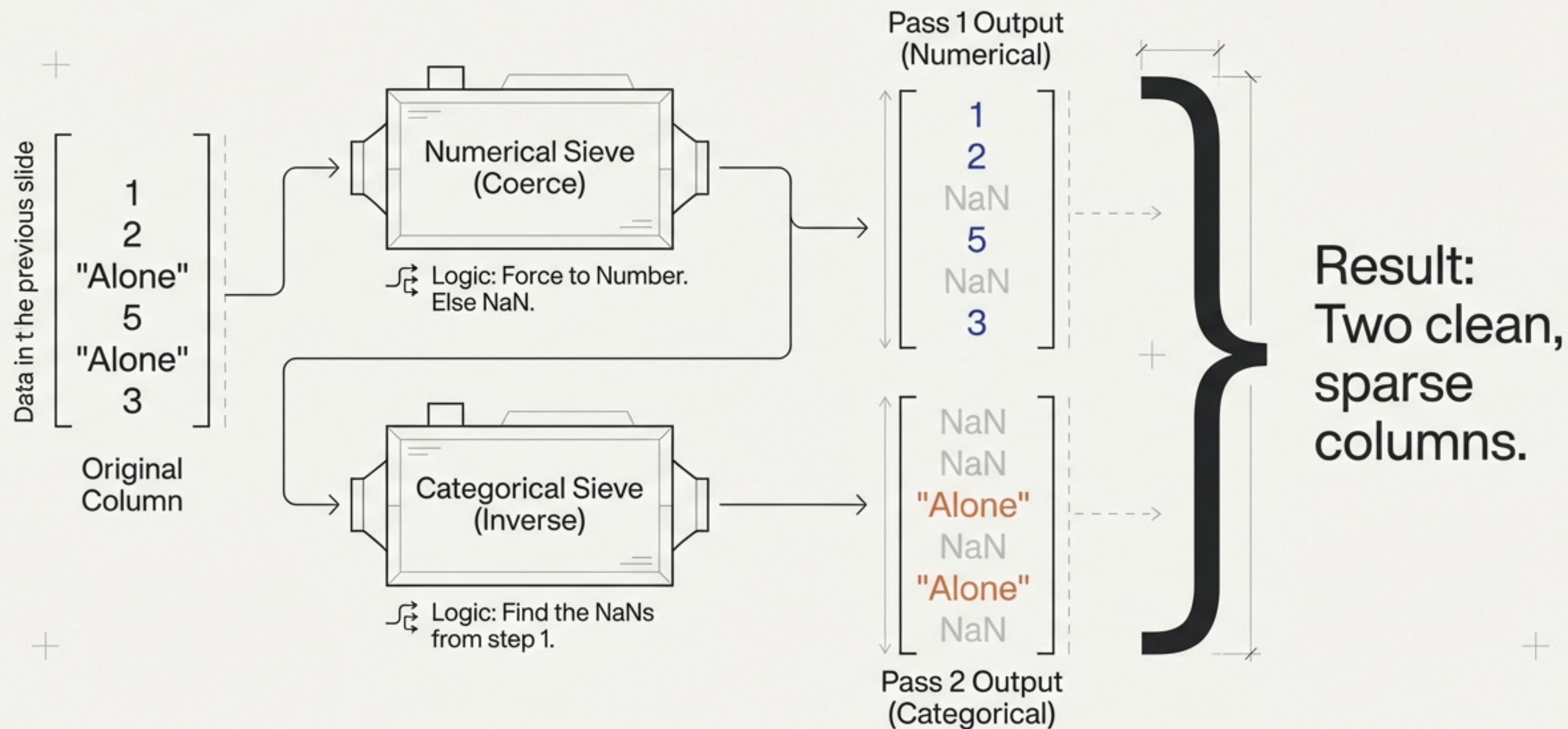
Because "Alone" is a string, Pandas forces the entire column to be Object type.

Mathematical operations (Mean, Sum) fail on the numbers 1, 2, 5 because they are technically strings.



The Strategy: Isolate and Coerce

Two-Pass Sieve



Technical Field Guide

Execution: Coercion Logic

Two-Pass Sieve: Implementing the Sieve in Python

```
# 1. Create Numerical Column
# errors='coerce' turns "Alone" into NaN
df['num_col'] = pd.to_numeric(df['mixed_col'], errors='coerce')

# 2. Create Categorical Column
# If num_col is null, grab the original string. Else NaN.
df['cat_col'] = np.where(df['num_col'].isnull(), df['mixed_col'],
                        np.nan)
```

The Secret Weapon.

Prevents crashes by converting errors to Missing Values.

Logic Switch:

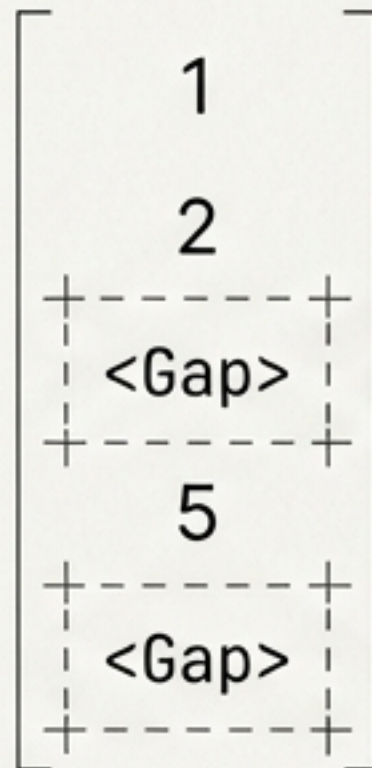
Where numeric is missing, keeping the string.

Handling the Aftermath: Imputation

Splitting creates gaps (NaNs). We must fill them.

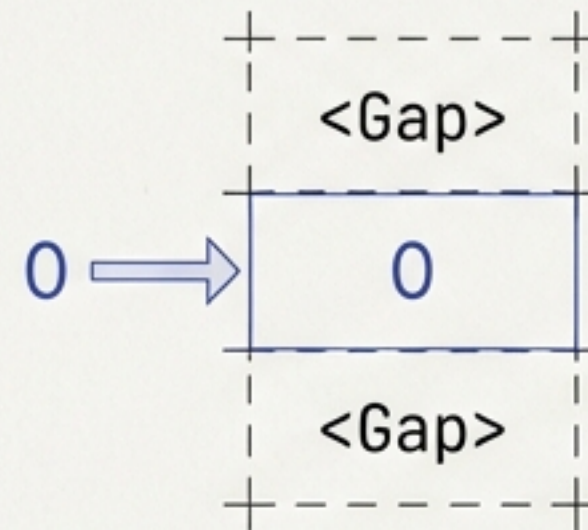
The Gaps

Visual of a column with holes:



Strategy for Numbers

Fill with 0 or Mean.

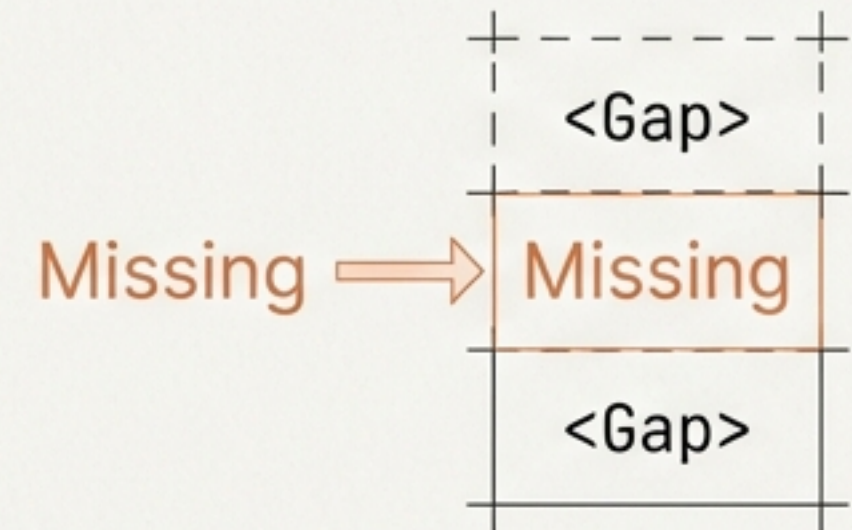


```
df['num'].fillna(0,  
inplace=True)
```

Implies 0 companions.

Strategy for Categories

Fill with "Missing".

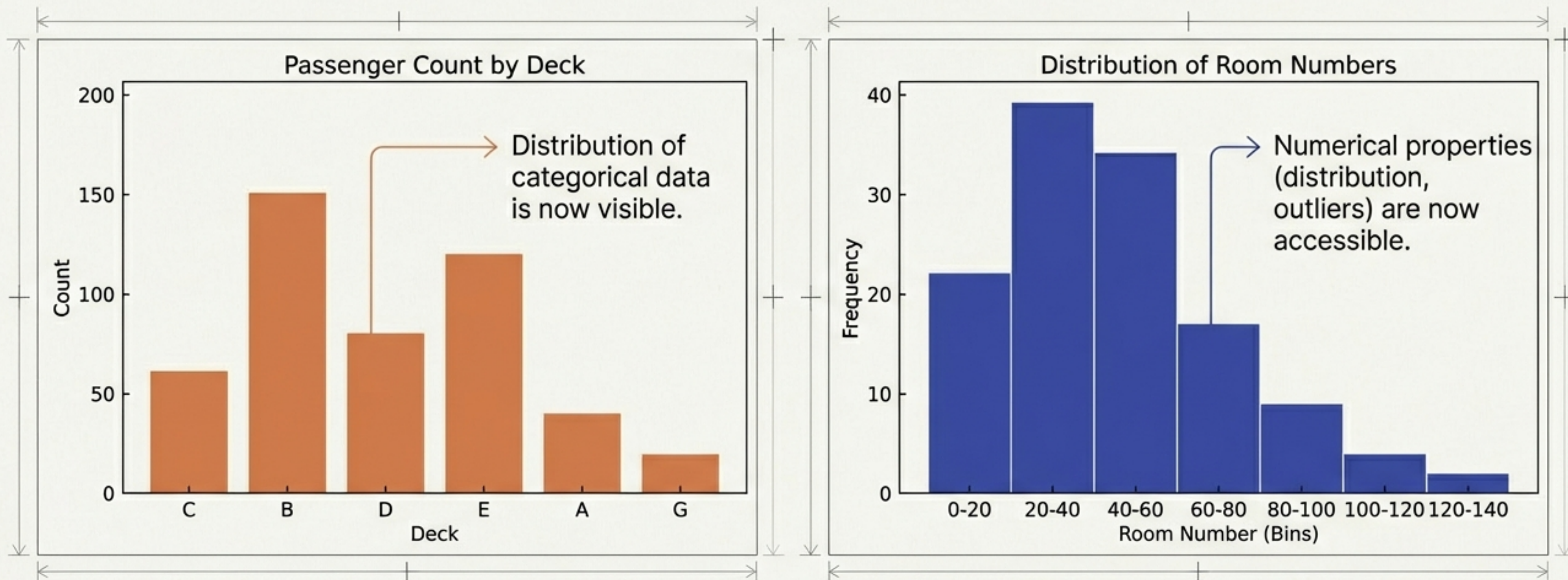


```
df['cat'].fillna('Missing',  
inplace=True)
```

Explicitly labels the absence.

The Result: Actionable Insights

Clean features unlock clear visualization.



We have successfully converted noise into signal.

Summary & Cheat Sheet

Scenario	Data Shape	Strategy	Key Pandas Function
The Fused Mix	Single String (e.g., "C85")	Vertical Split	<code>str.extract('(\d+)')</code>
The Interspersed Mix	Varying Rows (e.g., "1", "Alone")	Two-Pass Sieve	<code>pd.to_numeric(errors='coerce')</code>

Don't delete messy columns. Decode them.